

Aula 12 – Aritmética do Computador

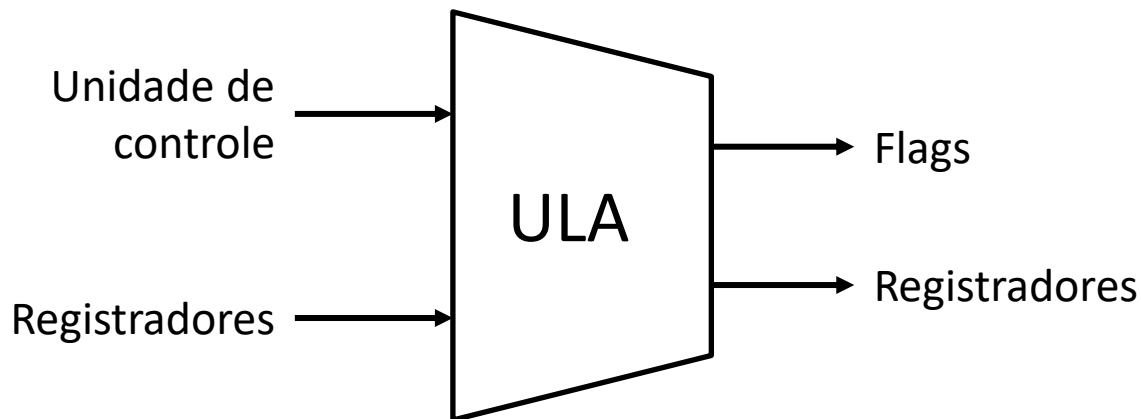
Prof. João Fernando Mari

joaofmari.github.io

- Unidade Aritmética e Lógica
- Representação de inteiros
 - Sinal-magnitude
 - Complemento de dois
 - Negação especial – caso 1
 - Negação especial – caso 2
 - Intervalo de números
 - Conversão entre tamanhos
- Adição e subtração
 - Hardware para adição e subtração
- Multiplicação
 - Exemplo de multiplicação
 - Multiplicação binária sem sinal
 - Execução do exemplo
 - Fluxograma - Multiplicação Binária Sem Sinal
 - Multiplicando números negativos
 - Algoritmo de Booth
 - Exemplo do algoritmo de Booth
- Divisão
 - Divisão de inteiros binários sem sinal
 - Fluxograma para divisão binária sem sinal
 - Tratando números negativos
 - EXEMPLO: $7/3$; $(-7)/3$; $7/(-3)$ e $(-7)/(-3)$

Unidade Aritmética e Lógica

- Responsável por realizar os cálculos.
- Tudo mais no computador existe para atender a essa unidade.
- Trata números inteiros.
- Pode tratar números de ponto flutuante (reais).



Representação de inteiros

- Somente 0's e 1's para representar tudo.
 - Números positivos armazenados em binário.
 - Ex: 41 = 00101001
 - Sem sinal de menos.
 - Sem ponto.

- Representações de números inteiros
 - Sinal-magnitude.
 - Complemento a dois.

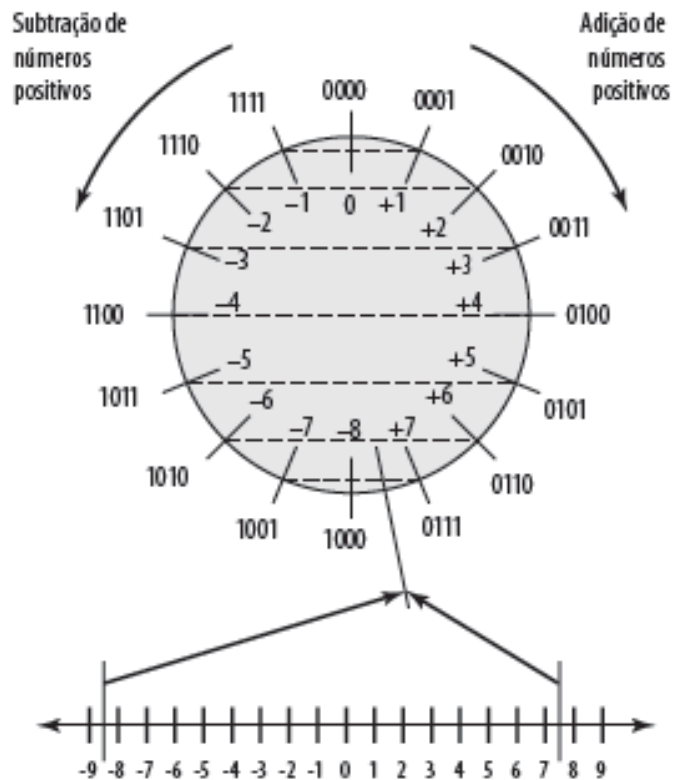
- Bit mais à esquerda é bit de sinal.
 - 0 significa positivo.
 - 1 significa negativo.
 - $+18 = 00010010$.
 - $-18 = 10010010$.
- Problemas:
 - Precisa considerar sinal e magnitude na aritmética.
 - Possui duas representações de zero (+0 e -0).
 - $+0 = 00000000$
 - $-0 = 10000000$

- $+3 = 00000011$
- $+2 = 00000010$
- $+1 = 00000001$
- $+0 = 00000000$
- $-1 = 11111111$
- $-2 = 11111110$
- $-3 = 11111101$

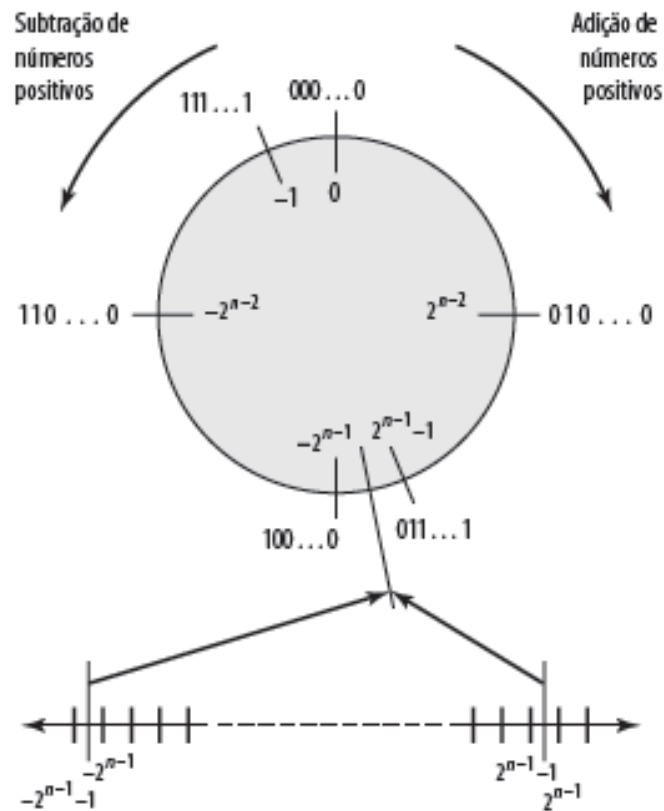
Complemento de dois

- Uma representação única do zero.
- Aritmética funciona com facilidade
 - Veremos mais adiante.
- A negação é muito fácil.
 - Ex.: $3 = 00000011$
 - Complemento booleano gera
 - 11111100
 - Somar 1 ao LSB (bit menos significativo)
 - 11111101
 - Ex.: $-3 = 11111101$
 - Complemento booleano gera
 - 00000010
 - Somar 1 ao LSB
 - 00000011

Representação geométrica dos inteiros de complemento a dois



(a) Números de 4 bits



(b) Números de n bits

Complemento de dois – Negação especial – caso 1

- 0 = 00000000
 - Not bit a bit:
 - 11111111
 - Some 1 ao LSB:
 - +1
 - Resultado:
 - 1 00000000
 - O estouro (overflow) é ignorado, portanto:
 - -0 = +0 → CERTO!

Complemento de dois – Negação especial – caso 2

- $-128 = 10000000$
 - Not bit-a-bit:
 - 01111111
 - Some 1 ao LSB:
 - +1
 - **Resultado:**
 - 10000000
 - Portanto:
 - $-(-128) = -128 \rightarrow$ ERRADO!
 - Monitorar o MSB (bit de sinal).
 - Ele deve mudar durante a negação.
- Quando somamos um valor negativo com um valor positivo é impossível ocorrer overflow.
 - O resultado tende a se aproximar de 0.
- Quando somamos dois valores com o mesmo sinal, devemos verificar se o bit de sinal muda:
 - EX: Somar dois números positivos (MSB=0) DEVE resultar em um número positivo (MSB=0).
 - EX: Somar dois números negativos (MSB=1) DEVE resultar em um número negativo (MSB=1).

Complemento de dois - Intervalo de números

- -2^{n-1} até $+2^{n-1}-1$
- Complemento a 2 com 8 bits:
 - $+127 = 01111111 = 2^7 - 1$
 - $-128 = 10000000 = -2^7$
- Complemento a 2 com 16 bits:
 - $+32767 = 01111111 11111111 = 2^{15} - 1$
 - $-32768 = 10000000 00000000 = -2^{15}$

Complemento de dois - Conversão entre tamanhos

- Pacote de número positivo com zeros iniciais.
 - 8 bits
 - +18 = 00010010
 - 16 bits
 - +18 = 00000000 00010010

- Pacote de números negativos com uns iniciais.
 - 8 bits
 - -18 = 11101110
 - 16 bits
 - -18 = 11111111 11101110

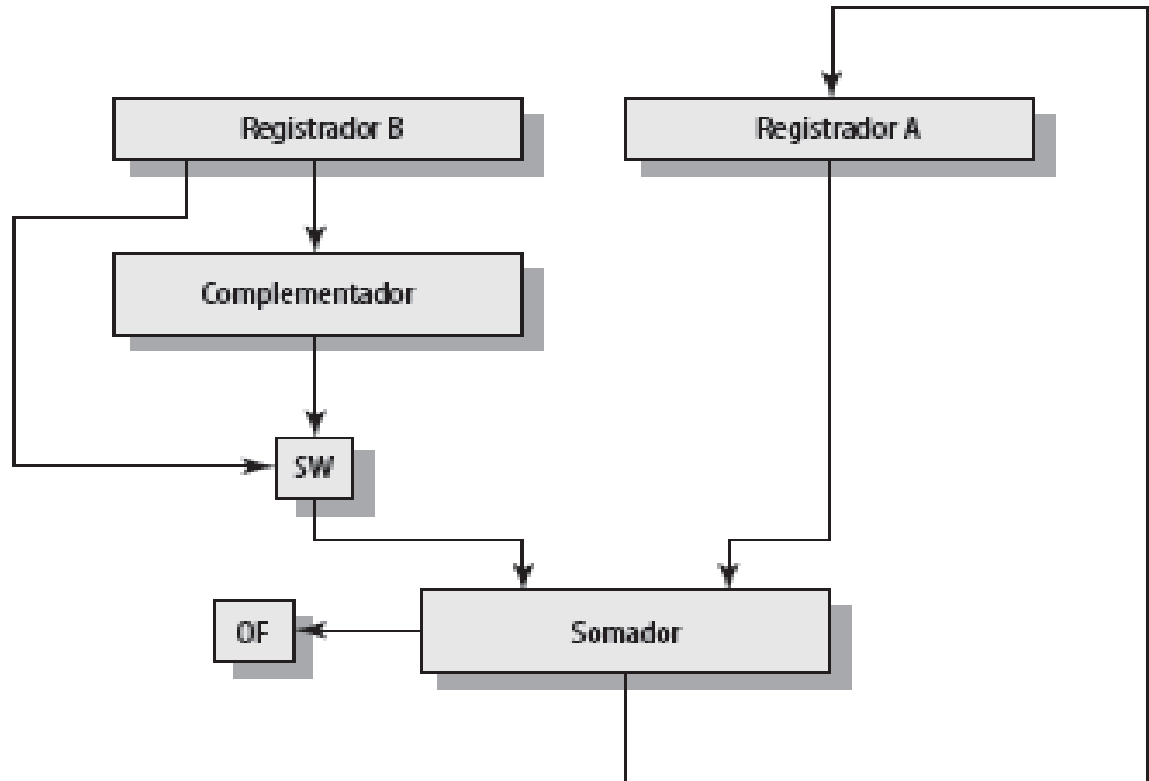
- Ou seja, pacote com o MSB (bit de sinal).
 - MSB – bit mais significativo.

ADIÇÃO E SUBTRAÇÃO

- Adição binária normal.
 - Monitore estouro no bit de sinal.
- Subtração
 - Pegue o complemento a dois do subtraendo e some ao minuendo.
 - Ou seja, $a - b = a + (-b)$.
 - Assim, só precisamos de circuitos de adição e complemento.

Hardware para adição e subtração

- add \$A, \$B



OF = bit de *overflow* (do inglês *overflow bit*)

SW = seletor – multiplexador (seleciona adição ou subtração)

Hardware para adição e subtração

- add \$A, \$B

- 4 bits

- ADIÇÃO:

- $2 + 3 = 5$

$0010 + 0011 = 0101$

- $(-2) + 3 = 1$

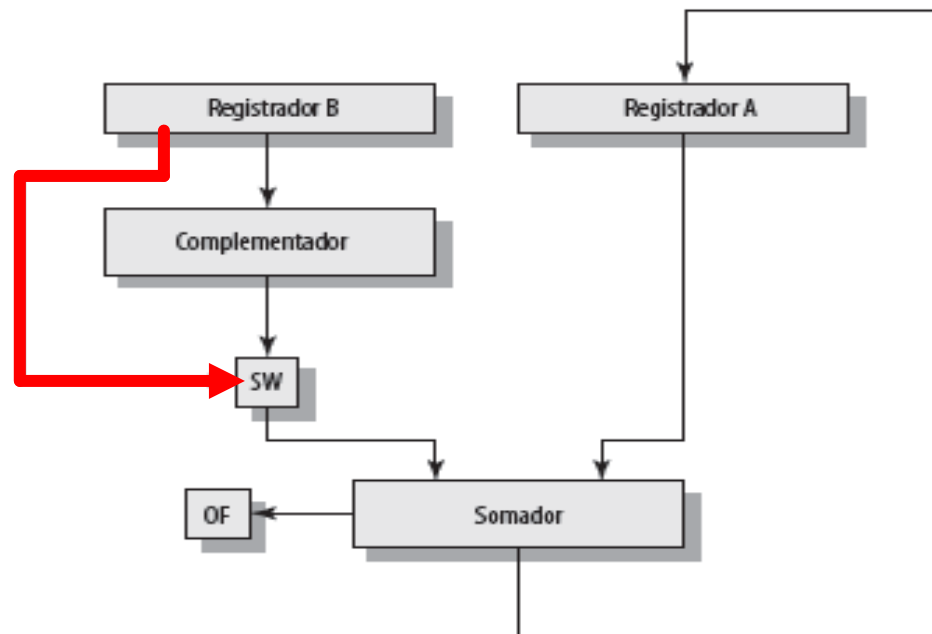
$1110 + 0011 = 0001$

- $2 + (-3) = -1$

$0010 + 1101 = 1111$

- $(-2) + (-3) = -5$

$1110 + 1101 = 1011$



OF = bit de *overflow* (do inglês *overflow bit*)

SW = seletor – multiplexador (seleciona adição ou subtração)

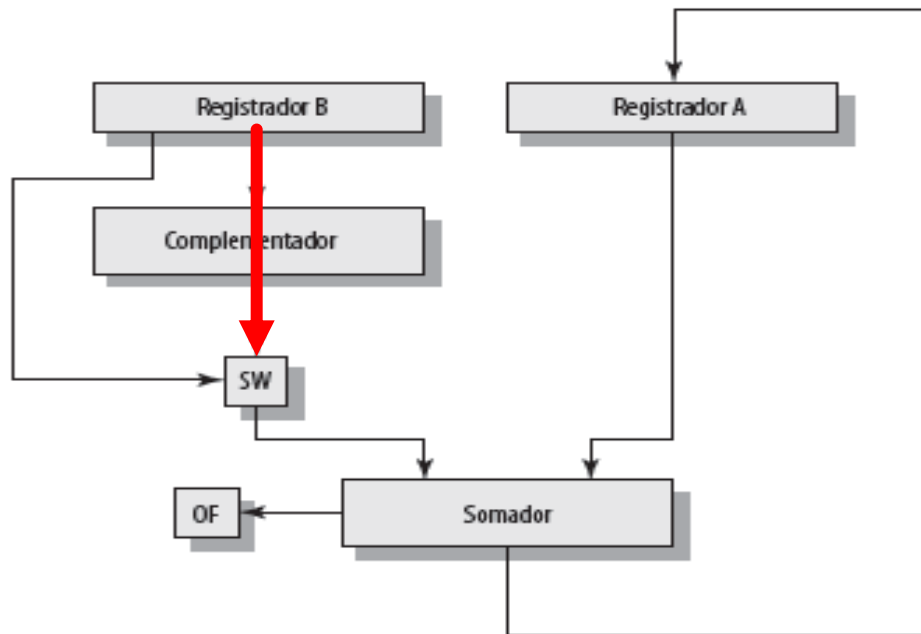
Hardware para adição e subtração

- add \$A, \$B

- 4 bits

- SUBTRAÇÃO:

- $2 - 3 = -1$
 - $2 + (-3)$ $0010 - 0011$
 $0010 + 1101 = 1111$
- $(-2) - 3 = -5$
 - $(-2) + (-3)$ $1110 - 0011$
 $1110 + 1101 = 1011$
- $2 - (-3) = 5$
 - $2 + 3$ $0010 - 1101$
 $0010 + 0011 = 0101$
- $(-2) - (-3) = 1$
 - $(-2) + 3$ $1110 - 1101$
 $1110 + 0011 = 0001$



OF = bit de *overflow* (do inglês *overflow bit*)

SW = seletor – multiplexador (seleciona adição ou subtração)

MULTIPLICAÇÃO

- Complexa.
- Calcule produto parcial para cada dígito.
- Cuidado com o valor da casa (coluna).
- Some produtos parciais.

Exemplo: Multiplicação

$$\begin{array}{r}
 \\
 \\
 \times \\
 \hline
 \\
 \\
 \\
 + \\
 \hline
 1
 \end{array}$$

Multiplicando (11_{10}) M

Multiplicador (13_{10}) Q

Produtos parciais

Produtos parciais

Produtos parciais

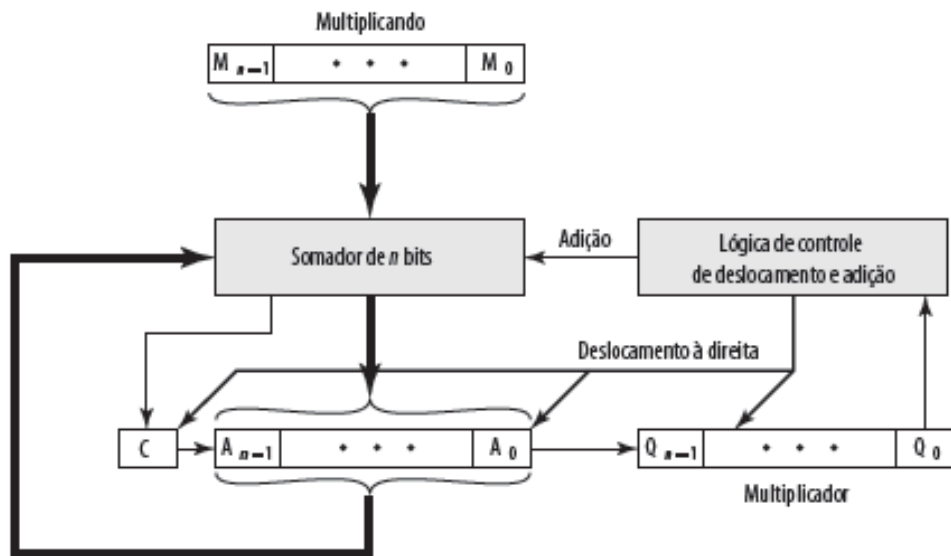
Produtos parciais

Produto (143_{10})

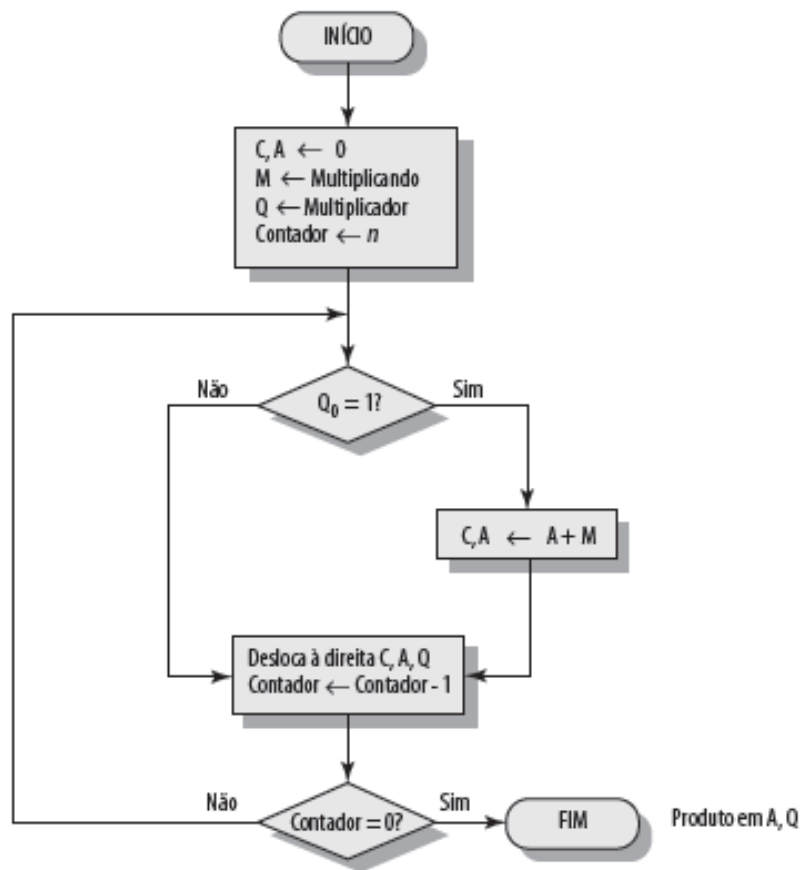
- Nota: Se o bit multiplicador for 1:
 - Copiar o multiplicando.
 - Caso contrário, zero.
- Nota: precisa de resultado com tamanho duplo.

Fluxograma - Multiplicação Binária Sem Sinal

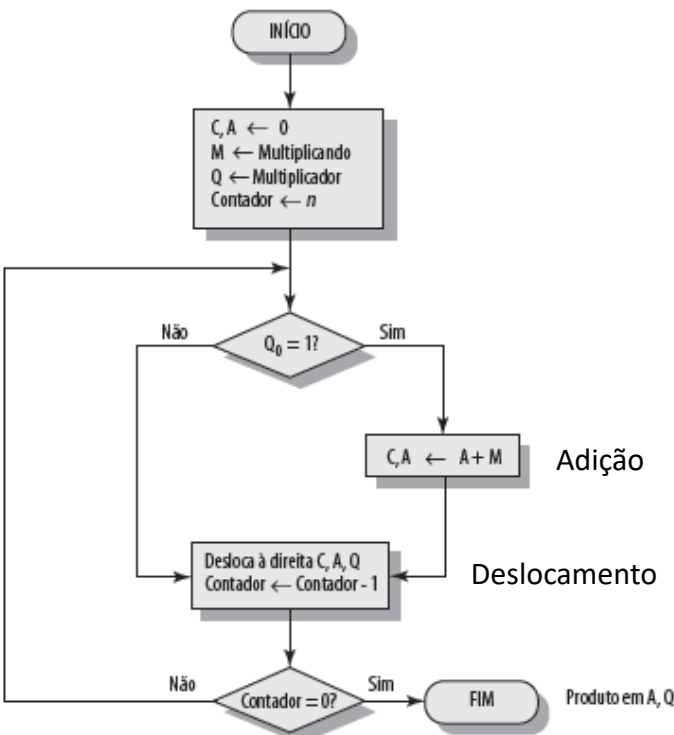
- $AQ = M \times Q$



(a) Diagrama em blocos



Execução do exemplo



11 × 13 = 143

	1	0	1	1	M
×	1	1	0	1	Q
	1	0	0	0	1 1 1 1 A Q

C	A	Q	M	
0	0000	1101	1011	Valores iniciais
0	1011	1101	1011	Adição } Primeiro ciclo
0	0101	1110	1011	
0	0010	1111	1011	Desl. } Segundo ciclo
0	1101	1111	1011	
0	0110	1111	1011	Adição } Terceiro ciclo
0	1101	1111	1011	
1	0001	1111	1011	Desl. } Quarto ciclo
0	1000	1111	1011	

Multiplicando números negativos

- Solução 1:
 - Converta para positivo, se for preciso.
 - Multiplique como antes.
 - Se sinais diferentes, negue a resposta.

- Exemplo:

- $3 \times 7 = 21;$

- $(-3) \times 7 = -21;$

- $3 \times (-7) = -21;$

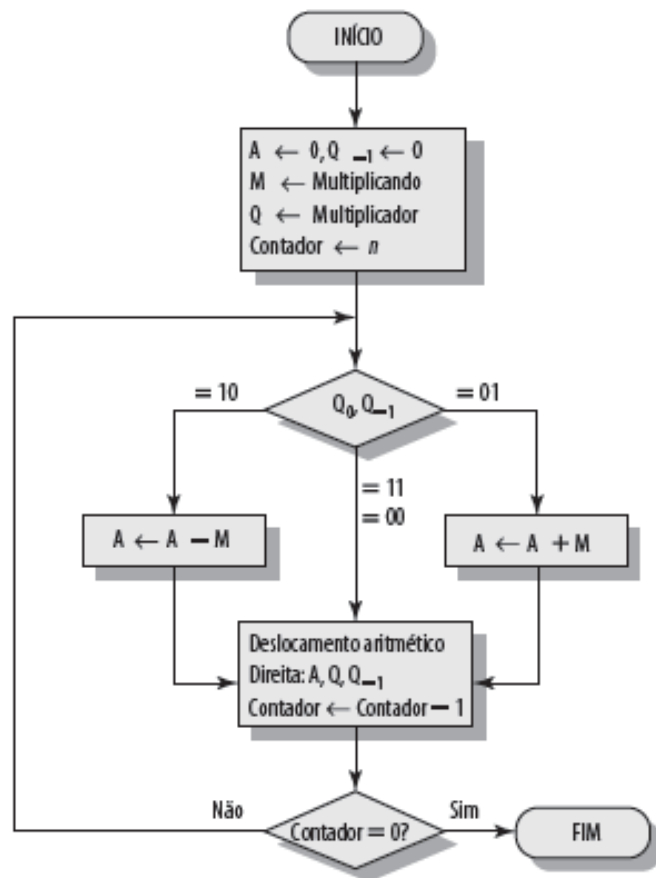
- $(-3) \times (-7) = 21;$

- Menos eficiente.

- Solução 2:
 - Algoritmo de Booth.

Algoritmo de Booth

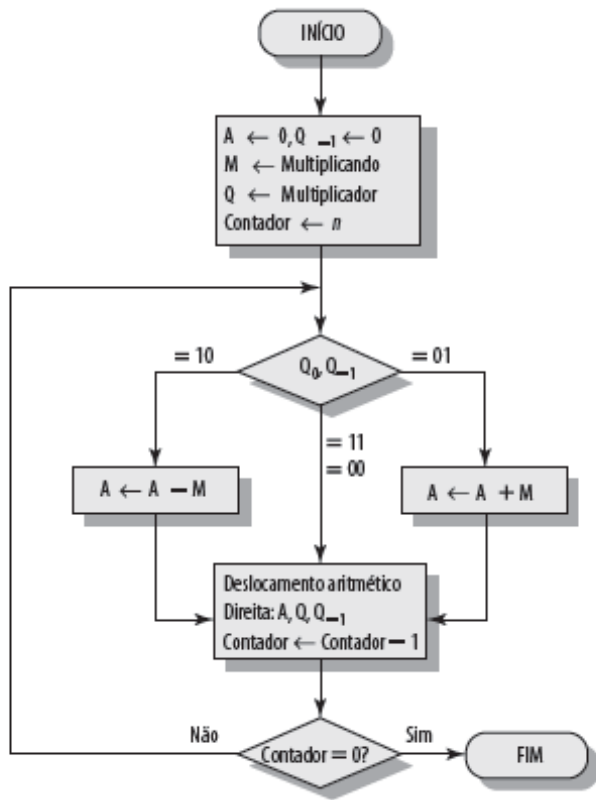
- Funciona com números inteiros em **complemento a dois**.
 - Deslocamento aritmético a direita.
 - O valor da posição mais a esquerda é mantido após o deslocamento.
 - *Ver conversão entre tamanhos para complemento de dois.*
 - Produto em A,Q



Exemplo: Algoritmo de Booth

7 × 3 = 21

	0	1	1	1	M
×	0	0	1	1	Q
	0	0	0	1	AQ



A	Q	Q ₋₁	M	
0000	0011	<u>0</u>	0111	Valores iniciais
1001	0011	0	0111	A ← A - M } Primeiro
1100	1001	<u>1</u>	0111	Deslocamento } ciclo
1110	0100	<u>1</u>	0111	Deslocamento } Segundo
0101	0100	1	0111	A ← A + M } Terceiro
0010	1010	<u>0</u>	0111	Deslocamento } ciclo
0001	0101	<u>0</u>	0111	Deslocamento } Quarto
				ciclo

DIVISÃO

- Mais complexa que a multiplicação.
- Baseada na divisão longa.

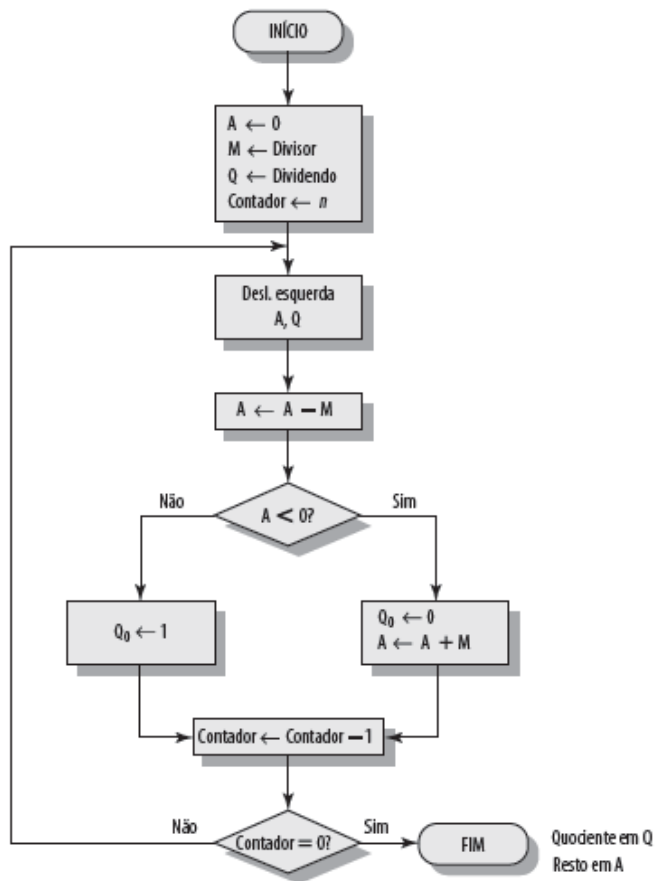
- Diagram illustrating the long division of 10110011 by 1011:

```

      0 0 0 0 1 1 0 1 ← Quociente (Q')
  1011 ) 10110011 ← Dividendo (Q)
        1011
        ---
         001110
          1011
          ---
           00111
            1011
            ---
             0011
              1011
              ---
               100 ← Resto (A)
  
```

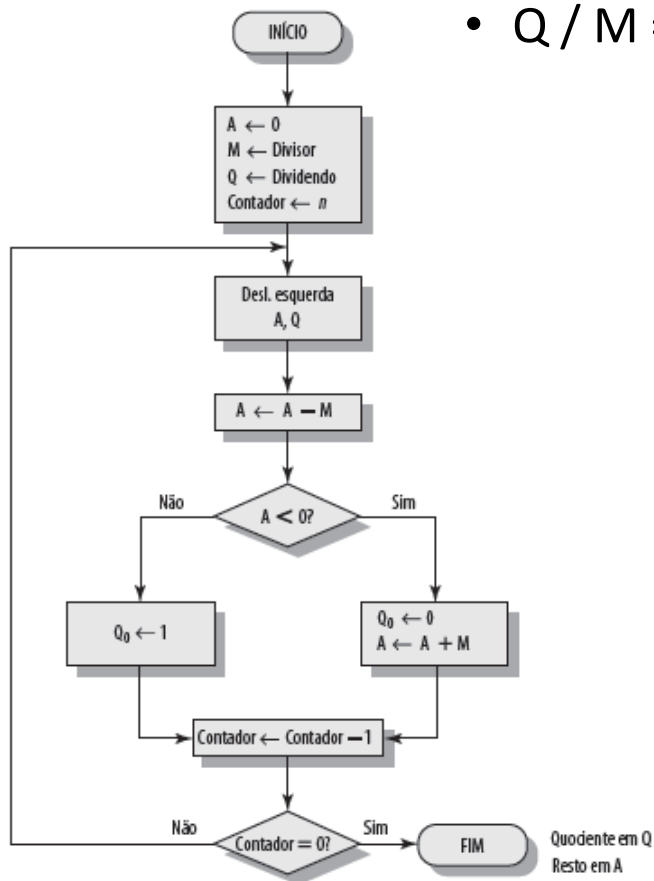
Fluxograma para divisão binária sem sinal

- $Q / M = Q' + A(R)$



Fluxograma para divisão binária sem sinal

$$Q / M = Q' + A(R)$$



A	Q	M = 0011
0000	0111	Valor inicial
0000	1110	Deslocar
1101		Subtrair
0000	1110	Restaurar
0001	1100	Deslocar
1110		Subtrair
0001	1100	Restaurar
0011	1000	Deslocar
0000		Subtrair
0000	1001	Fazer Q ₀ = 1
0001	0010	Deslocar
1110		Subtrair
0001	0010	Restaurar

(a) (7) ÷ (3)

Tratando números negativos

- $Q/M = Q' + A(R)$
 - Dividendo / Divisor = Quociente + Resto
- Suponha todas as combinações possíveis de sinais de Q e M:
 - $Q = 7 \quad M = 3 \quad \rightarrow \quad Q' = 2 \quad A = 1$
 - $Q = 7 \quad M = -3 \quad \rightarrow \quad Q' = -2 \quad A = 1$
 - $Q = -7 \quad M = 3 \quad \rightarrow \quad Q' = -2 \quad A = -1$
 - $Q = -7 \quad M = -3 \quad \rightarrow \quad Q' = 2 \quad A = -1$

Exemplo: $7/3$; $(-7)/3$; $7/(-3)$ e $(-7)/(-3)$

- Algoritmo de divisão por restauração:

- Para realizar a divisão com operandos com sinal (complemento a dois):

- Converta os operandos (Q e M) em valores sem sinal.
 - Para isso tome o complemento de 2 dos números negativos.
- Realize a divisão utilizando o algoritmo de divisão para números sem sinal.
- Derive o sinal de Q' e de $A(R)$ a partir dos sinais de Q e M.

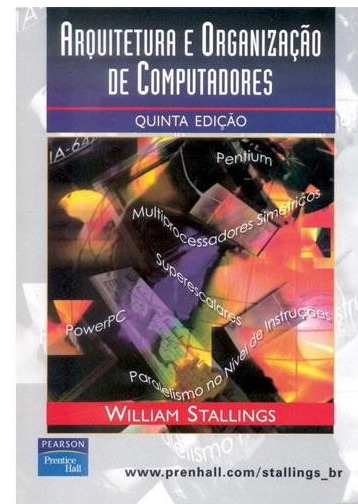
$$Q / M = Q' + A(R)$$

A	Q	M = 0011
0000	0111	Valor inicial
0000	1110	Deslocar
1101		Subtrair
0000	1110	Restaurar
0001	1100	Deslocar
1110		Subtrair
0001	1100	Restaurar
0011	1000	Deslocar
0000		Subtrair
0000	1001	Fazer $Q_0 = 1$
0001	0010	Deslocar
1110		Subtrair
0001	0010	Restaurar

(a) $(7) \div (3)$

Referências

- STALLINGS, W. **Arquitetura e Organização de Computadores**, 8. Ed., Pearson, 2010.
 - **Capítulo 9**



FIM